

Covergroups en Verificación Funcional

Tecnológico de Costa Rica - Curso de Verificación Funcional

Fabián Chacón Solano - 2018135154

I. ¿QUÉ ES UN COVERGROUP EN VERIFICACIÓN FUNCIONAL?

Un *covergroup* es una construcción de SystemVerilog utilizada para medir cobertura funcional. Su propósito es registrar qué valores, eventos o combinaciones relevantes del diseño han sido ejecutados durante una simulación. A diferencia de la cobertura de código, que indica qué partes de la implementación fueron ejecutadas, la cobertura funcional está relacionada con la intención del diseño y con el plan de verificación [1], por ende es una herramienta clave para evaluar el progreso de la verificación y asegurar que se están evaluando los escenarios.

En otras palabras, un *covergroup* permite responder preguntas como: “¿se probaron todos los tipos de operación?”, “¿se ejercitaron los casos borde?” o “¿se observaron las combinaciones importantes de entradas?”. Spear explica que la cobertura funcional ayuda a medir el progreso de la verificación y a encontrar huecos de cobertura que requieren nuevos estímulos o ajustes en las pruebas [1], este trabajo es fundamental puesto que humanamente es considerablemente complejo cubrir todos los escenarios posibles, por lo que el uso de covergroups es una herramienta relevante para el diseño funcional de circuitos integrados.

II. ¿QUÉ PARTES TIENE UN COVERGROUP?

Un *covergroup* está formado principalmente por los siguientes elementos:

- **Covergroup:** bloque que agrupa una o más mediciones de cobertura. Puede ser muestreado explícitamente con el método `sample()` o mediante un evento declarado en su definición.
- **Coverpoints:** variables o expresiones que se desean observar. Cada *coverpoint* mide qué valores de una señal, variable o expresión fueron ejecutados durante la simulación [1].
- **Bins:** contadores asociados a valores o rangos de valores. Los *bins* son la unidad básica de medición de cobertura funcional, cuando se muestrea un valor que cae dentro de un *bin*, ese bin queda cubierto [1].
- **Cross coverage:** medición de combinaciones entre dos o más *coverpoints*. Se usa cuando no basta con saber que dos valores aparecieron por separado, sino que es requerido para comprobar que aparecieron en combinación [1].
- **Opciones de cobertura:** parámetros como `option.per_instance`, `option.goal`, `option.weight` o límites para bins automáticos. Estas opciones permiten controlar cómo se calcula y reporta la cobertura [1].

- **Bins especiales:** SystemVerilog permite usar `ignore_bins` para excluir valores que no deben afectar el cálculo, también `illegal_bins` para marcar valores que representan situaciones inválidas [1].

III. ¿POR QUÉ ES IMPORTANTE LA IMPLEMENTACIÓN DE COVERGROUPS?

La implementación de *covergroups* es importante porque permite medir de manera explícita si el testbench realmente está verificando los escenarios definidos en el plan de verificación. Sin cobertura funcional, una simulación puede terminar sin errores y aun así no haber probado casos importantes del diseño.

Spear resalta que la cobertura funcional se usa dentro de un ciclo de retroalimentación, o sea se ejecutan pruebas, se analiza la cobertura alcanzada, se identifican huecos y luego se modifican los estímulos o restricciones para cubrir escenarios faltantes [1]. Esto permite mejorar la calidad del proceso de verificación y tomar decisiones basadas en datos, no solo en la cantidad de pruebas ejecutadas.

Además, los *covergroups* ayudan a diferenciar entre haber ejecutado código y haber verificado funcionalidad. Un diseño puede alcanzar alta cobertura de código, pero seguir sin probar ciertas condiciones funcionales importantes. Por eso, la cobertura funcional complementa la cobertura de código y permite evaluar el avance frente a la especificación [1].

IV. RECOMENDACIONES PARA LA CREACIÓN DE COVERGROUPS

Para crear *covergroups* útiles se recomiendan las siguientes prácticas:

- **Partir del plan de verificación.** Los *coverpoints* deben representar comportamientos, valores y combinaciones importantes para la especificación, no señales escogidas al azar.
- **Definir bins significativos.** Para señales grandes, no es conveniente depender siempre de bins automáticos. Es preferible definir rangos útiles, casos borde y valores especiales, como ceros, máximos, mínimos o estados relevantes [1].
- **Evitar cobertura decorativa.** Un *covergroup* debe medir algo que ayude a tomar decisiones de verificación. Si una métrica no cambia la estrategia de pruebas, probablemente no aporta valor.
- **Muestrear en el momento correcto.** La cobertura debe muestrearse cuando los datos sean válidos, por ejemplo al completarse una transacción o cuando el monitor observa una salida estable. Spear indica que las dos

partes principales de la cobertura funcional son los datos muestreados y el momento en que se muestrean [1].

- **Deshabilitar cobertura durante reset.** Se recomienda evitar que valores de inicialización o reset contaminen la cobertura. Esto puede hacerse con condiciones como `iff (!reset)` [1].
- **Usar cross coverage con cuidado.** La cobertura cruzada puede crecer rápidamente, ya que el número de bins aumenta con las combinaciones. Por eso debe usarse solo cuando la relación entre variables sea realmente importante [1].
- **Usar ignore_bins e illegal_bins cuando corresponda.** Los valores imposibles o irrelevantes deben excluirse del cálculo, mientras que los valores inválidos deben marcarse como ilegales.
- **Analizar los reportes de cobertura.** La cobertura no debe ser solo un número final. Deben revisarse los bins sin cubrir para decidir si se necesitan más datos aleatorios, pruebas dirigidas o cambios en las restricciones.

REFERENCIAS

- [1] C. Spear, *SystemVerilog for Verification: A Guide to Learning the Testbench Language Features*. New York, NY, USA: Springer Science+Business Media, 2008.